Assignment 3 — Bridge Pattern

Topic A — Drawing

Student: Yerassyl Makhanbetali

Group: SE-2522

1. Bridge Pattern

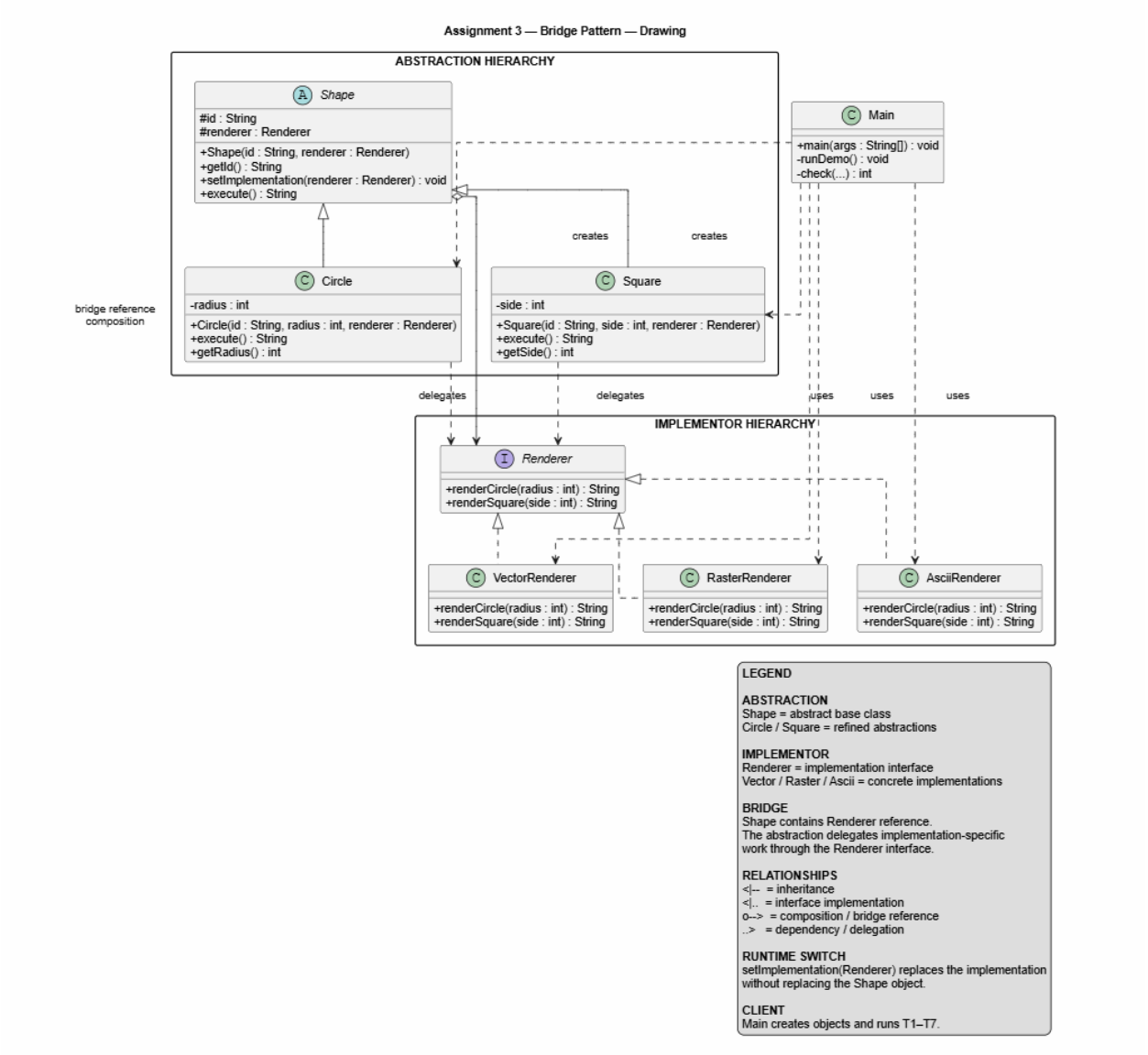
This project demonstrates the Bridge design pattern using the Drawing topic.

The abstraction hierarchy consists of Shape, Circle, and Square.
The implementation hierarchy consists of Renderer, VectorRenderer, RasterRenderer, and AsciiRenderer.

The two hierarchies are connected through composition.
Shape stores a Renderer interface reference and delegates rendering operations to it.

Bridge is suitable because the abstraction and implementation can vary independently. New renderers can be added without changing Circle or Square.

2. UML Diagram



Code + T1–T7

3. Bridge Implementation

The Shape class contains a Renderer reference. The Renderer object is supplied through the Shape constructor.

Circle and Square do not create concrete renderer objects.

Instead, they call renderer operations through the Renderer interface.

The setImplementation() method allows the implementation to be changed at runtime.

```
public abstract class Shape {  6 usages  2 inheritors  aintrealera
    protected final String id;  2 usages
    protected Renderer renderer;  4 usages

    public Shape(String id, Renderer renderer) {  6 usages  aintrealera
        this.id = id;
        this.renderer = renderer;
    }

    public String getId() {  1 usage  aintrealera
        return id;
    }

    public void setImplementation(Renderer renderer) {  1 usage  aintrealera
        this.renderer = renderer;
    }

    public abstract String execute();  8 usages  2 implementations  aintrealera
}
```

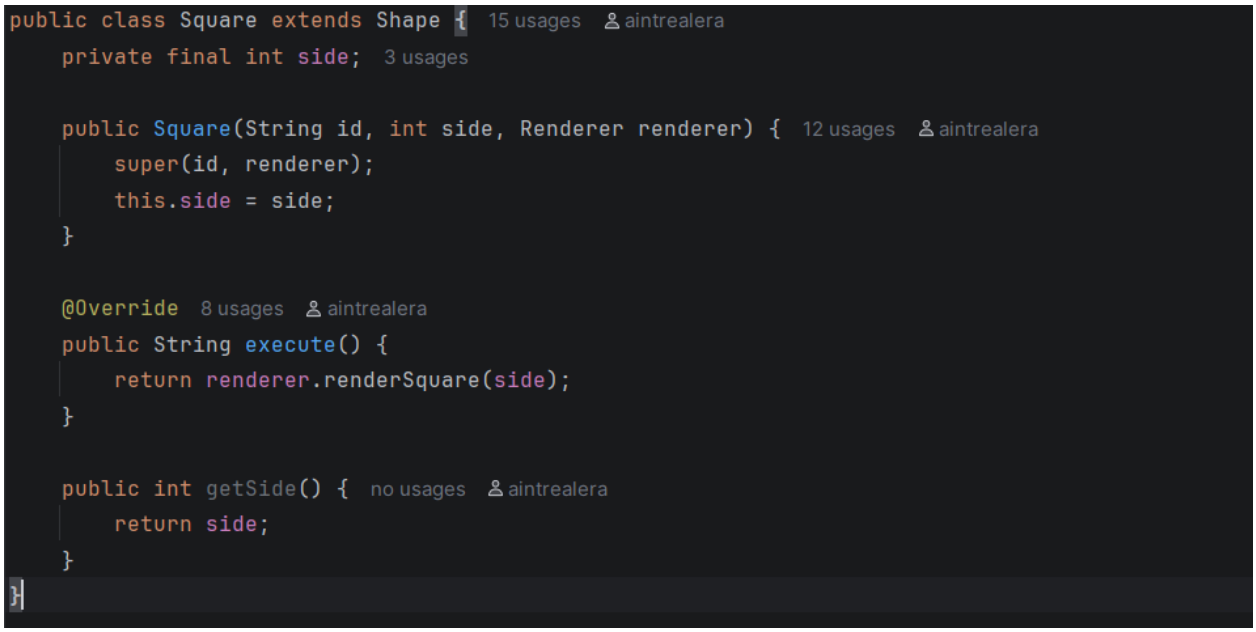


```java
public class Circle extends Shape {  19 usages  aintrealera
    private final int radius;  3 usages

    public Circle(String id, int radius, Renderer renderer) {  14 usages  aintrealera
        super(id, renderer);
        this.radius = radius;
    }

    @Override  8 usages  aintrealera
    public String execute() {
        return renderer.renderCircle(radius);
    }

    public int getRadius() {  1 usage  aintrealera
        return radius;
    }
}
```



```java
public class Square extends Shape {  15 usages  aintrealera
    private final int side;  3 usages

    public Square(String id, int side, Renderer renderer) {  12 usages  aintrealera
        super(id, renderer);
        this.side = side;
    }

    @Override  8 usages  aintrealera
    public String execute() {
        return renderer.renderSquare(side);
    }

    public int getSide() {  no usages  aintrealera
        return side;
    }
}
```

4. Clean Code:

Separation of responsibilities:

Shape stores domain data while Renderer implementations perform rendering.

Meaningful names:

Class names clearly describe their roles.

Focused methods:

execute(), renderCircle(), and renderSquare() perform small specific tasks.

Avoiding duplication:

Both Circle and Square use the same Renderer interface.

Encapsulation:

Shape stores the ID and renderer while Circle and Square store their own dimensions.

5. Test Results

```
T1 PASS | Circle + VectorRenderer | result=VECTOR circle radius=2
T2 PASS | Circle + RasterRenderer | result=RASTER circle radius=2
T3 PASS | Square + VectorRenderer | result=VECTOR square side=3
T4 PASS | Square + RasterRenderer | result=RASTER square side=3
T5 PASS | runtime switch | result=sameObject=true | stateUnchanged=true | before=VECTOR circle radius=2 | after=RASTER circle radius=2
T6 PASS | Circle + AsciiRenderer | result=ASCII circle radius=2
T7 PASS | Square + AsciiRenderer | result=ASCII square side=3
SUMMARY: 7/7 PASS

Process finished with exit code 0
```

6. Runtime Switching

T5 demonstrates runtime switching.

The same Circle object is first executed with VectorRenderer.

Then setImplementation() replaces the Renderer reference with RasterRenderer.

The Circle object remains the same and its ID and radius remain unchanged. Only the implementation-specific result changes.

```
Circle circle5 = new Circle( id: "C5",  radius: 2, vector);
Circle original = circle5;

String before = circle5.execute();

circle5.setImplementation(raster);

String after = circle5.execute();

boolean sameObject = original == circle5;
```

7. Independent Implementation

AsciiRenderer was added as the third implementation.

It implements the existing Renderer interface and supports both

Circle and Square.

The existing Shape classes, Renderer interface, VectorRenderer,

and RasterRenderer do not need to be changed to support the new

implementation.

This demonstrates the extensibility of the Bridge pattern.

```
public class AsciiRenderer implements Renderer {  8 usages   aintrealera

    @Override  1 usage   aintrealera
    public String renderCircle(int radius) {
        return "ASCII circle radius=" + radius;
    }

    @Override  1 usage   aintrealera
    public String renderSquare(int side) {
        return "ASCII square side=" + side;
    }
}
```

8. Bridge vs Adapter

Bridge separates two independently varying dimensions and connects

them through composition.

In this project, the two dimensions are Shapes and Renderers.

Adapter has a different intent. It makes an existing incompatible

interface work with another interface expected by the client.

Therefore, Bridge is appropriate here because the goal is to keep

the Shape and Renderer hierarchies independent.

9. Trade-off

One trade-off of Bridge is additional design complexity because it introduces an abstraction and an implementation interface.

However, this reduces coupling and makes it easier to add or replace implementations without creating a subclass for every Shape and Renderer combination.